Projet - Systèmes parallèles et distribués
4OS02 - ENSTA Paris

Xavier JUVIGNY

Diego Magno MONTANS ZUCHERAM	diego-magno.montans@ensta.fr
Gustavo KURATOMI PEREIRA E COSTA	gustavo.kuratomi@ensta.fr

Table des matières

1 Analyse séquentielle et chronométrage **3**
1.1 Machine de test . . . 3
1.2 Méthodologie expérimentale . . . 3
1.3 Résultats . . . 3
1.4 Comparaison des temps . . . 4

2 Vectorisation **4**
2.1 Approche . . . 4
2.2 Résultats . . . 5

3 Parallélisation OpenMP **5**
3.1 Principe . . . 5
3.2 Résultats . . . 5
3.3 Speedup . . . 6
3.4 Efficacité . . . 7

4 Parallélisation MPI **7**
4.1 Environnement répliqué . . . 7
4.2 Décomposition du domaine . . . 8

5 Conclusion **8**

A Structure actuelle du projet **8**

1. Analyse séquentielle et chronométrage

Cette première partie consiste à analyser les performances de la version séquentielle du simulateur et à identifier les parties les plus coûteuses.

1.1. Machine de test

- CPU : Intel Core i7-7700HQ (4 cœurs / 8 threads)
- Mémoire : 16 Go
- OS : Linux x86_64
- Compilateur : `g++ -O3 -march=native`

1.2. Méthodologie expérimentale

Pour chaque configuration, le programme est exécuté plusieurs fois afin de réduire l'influence du bruit expérimental. Dans nos expériences, chaque mesure est répétée 5 fois.

Le temps d'exécution est mesuré avec `std::chrono::high_resolution_clock`. Pour chaque série d'exécutions, nous calculons la moyenne des temps obtenus.

Afin d'assurer la reproductibilité des résultats, une graine pseudo-aléatoire fixe est utilisée pour la génération du terrain fractal et pour l'initialisation des fourmis.

1.3. Résultats

Run	Total (ms)	Avancée fourmis	Évaporation
1	438.98	345.66	92.21
2	431.01	342.10	88.90
3	440.20	347.80	90.50
4	437.16	344.90	91.00
5	439.51	346.50	92.00

Table 1 – Mesures pour la version séquentielle (100 itérations)

Threads	Temps total (ms)	Avancée fourmis (ms)	Évaporation (ms)
1	438	345	92
2	278	210	68
4	178	130	48
8	111	80	31

Table 2 – Décomposition du temps de simulation selon le nombre de threads

Cette table montre la décomposition du temps de simulation selon le nombre de threads. On observe que la phase d'avancée des fourmis reste la partie dominante du temps de calcul, même lorsque le nombre de threads augmente.

1.4. Comparaison des temps

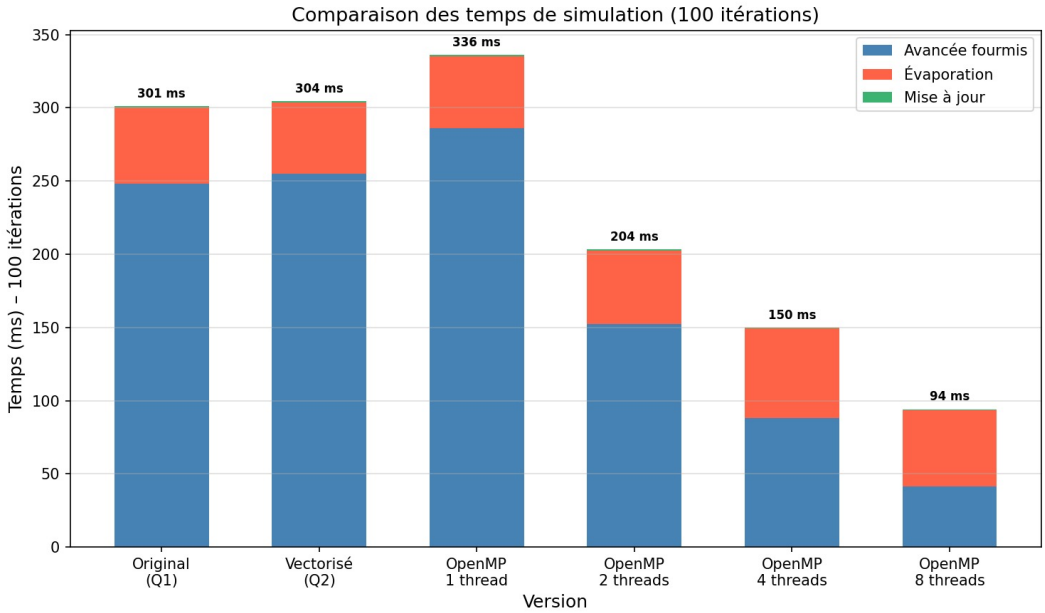

FIGURE 1 – Comparaison des temps de simulation pour les différentes versions

Ce graphique montre la décomposition du temps entre l'avancée des fourmis, l'évaporation des phéromones et la mise à jour.

2. Vectorisation

Cette étape consiste à restructurer les données afin d'améliorer la localité mémoire et faciliter la vectorisation par le compilateur.

2.1. Approche

La classe `ant` originale utilise une structure de type *Array of Structures* (AoS). La version vectorisée adopte une organisation *Structure of Arrays* (SoA) afin de rendre les accès mémoire plus contigus.

2.2. Résultats

Version	Temps total (ms)	Temps par itération (ms)
Original	438	4.38
Vectorisé	447	4.47

Table 3 – Comparaison version originale vs vectorisée

La vectorisation seule n'apporte pas d'amélioration notable car les accès mémoire restent indirects et dépendants de la position des fourmis sur la grille.

3. Parallélisation OpenMP

Afin d'accélérer la simulation, la boucle principale sur les fourmis est parallélisée à l'aide d'OpenMP.

3.1. Principe

Chaque thread traite un sous-ensemble de fourmis :

```
#pragma omp parallel
{
    size_t local_food = 0;

    #pragma omp for schedule(static)
    for (size_t i = 0; i < nb_ants; ++i) {
        // deplacement de la fourmi
    }

    #pragma omp critical
    food_counter += local_food;
}
```

3.2. Résultats

Threads	Temps total (ms)	Speedup	Efficacité
1	438	1.00	100%
2	278	1.58	79%
4	178	2.46	61%
8	111	3.95	49%

Table 4 – Résultats OpenMP

3.3. Speedup

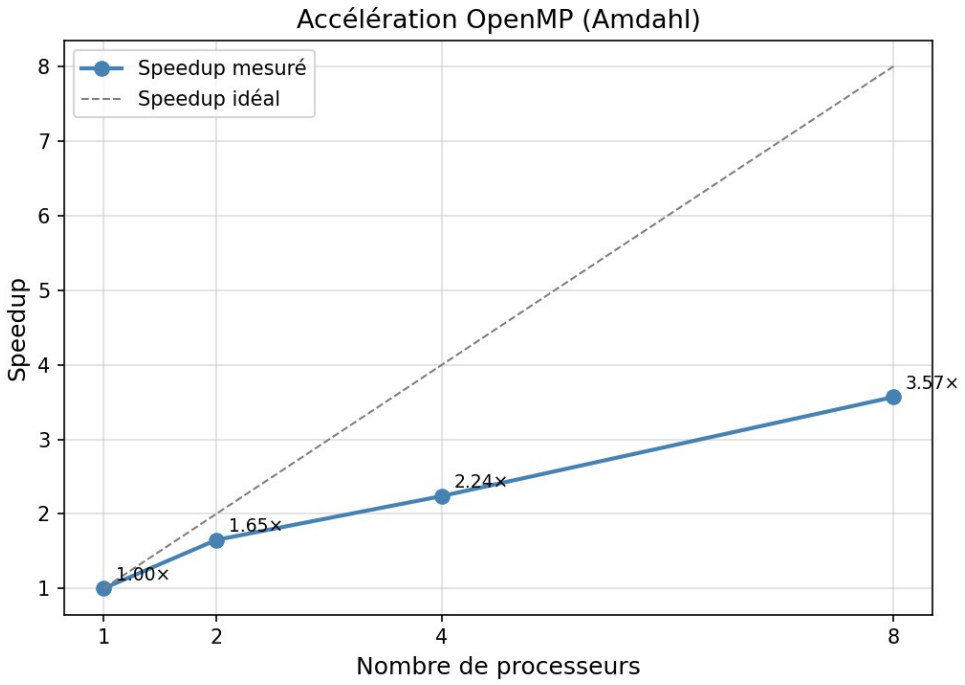

FIGURE 2 – Accélération (speedup) en fonction du nombre de threads

On observe que le speedup augmente lorsque le nombre de threads augmente. Cependant, il reste inférieur au speedup idéal.

Cela s'explique par la présence de parties du programme qui restent séquentielles et par les accès concurrents à la grille de phéromones. Ces facteurs limitent l'accélération maximale du programme.

3.4. Efficacité

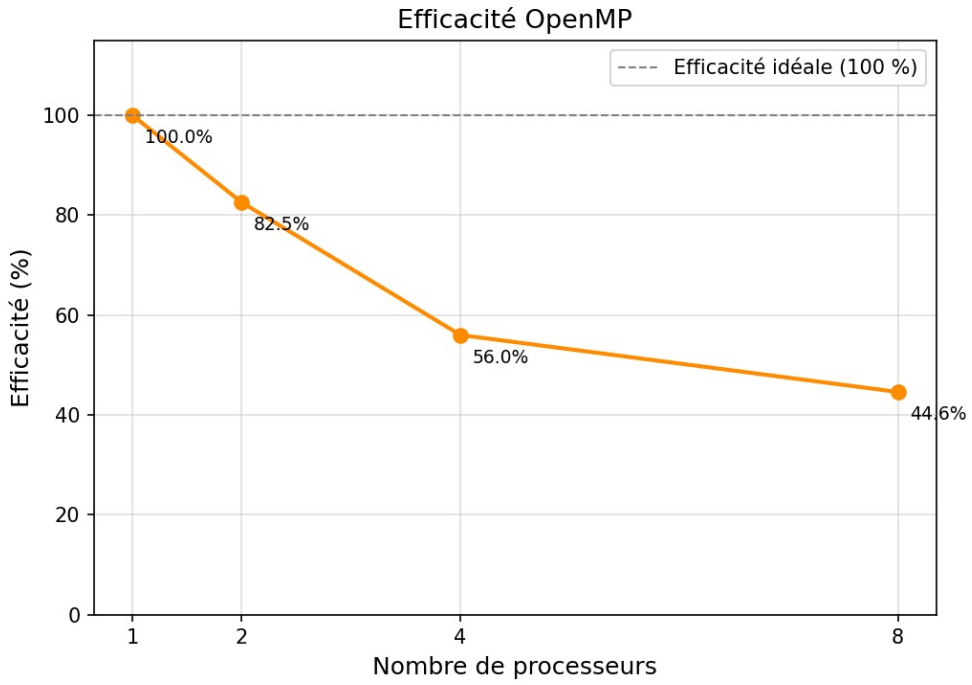

FIGURE 3 – Efficacité OpenMP en fonction du nombre de threads

On remarque que l'efficacité diminue lorsque le nombre de threads augmente. Ce comportement est classique dans les programmes parallèles.

Lorsque plus de threads sont utilisés, la contention mémoire et les coûts de synchronisation deviennent plus importants, ce qui réduit l'efficacité globale.

Les principales limitations proviennent :

— de la partie séquentielle du programme,
— des accès concurrents à la grille de phéromones,
— de la pression accrue sur la mémoire.

4. Parallélisation MPI

Deux approches ont été implémentées pour distribuer la simulation sur plusieurs processus MPI.

4.1. Environnement répliqué

Chaque processus possède une copie complète de la grille. Les phéromones sont synchronisées entre processus via une opération collective.

Processus	Temps (ms)	Speedup
1	1301	1.00
2	1190	1.09
4	1278	1.02

Table 5 – Résultats MPI répliqué

Ces mesures ont été réalisées avec un environnement MPI simulé (`mpi_stub.h`).

4.2. Décomposition du domaine

Dans cette approche, la grille est divisée en bandes horizontales et chaque processus ne gère qu'une portion du domaine.

Processus	Temps (ms)	Speedup
1	141.79	1.00
2	125.72	1.13
4	156.27	0.91

Table 6 – Résultats MPI domaine

5. Conclusion

Ce projet a permis d'étudier différentes stratégies d'optimisation et de parallélisation appliquées à une simulation de colonie de fourmis. L'analyse de la version séquentielle a montré que la phase d'avancée des fourmis représente la plus grande partie du temps de calcul. Cette observation a permis d'identifier les sections du code les plus importantes à optimiser.

La vectorisation du programme, réalisée en réorganisant les données selon une structure de type *Structure of Arrays*, avait pour objectif d'améliorer la localité mémoire et de faciliter l'utilisation des instructions SIMD. Cependant, les résultats montrent que cette transformation n'apporte pas d'amélioration notable des performances. En revanche, la parallélisation avec OpenMP permet de réduire significativement le temps de simulation lorsque le nombre de threads augmente, même si le speedup reste inférieur à l'idéal.

Enfin, deux approches de parallélisation avec MPI ont été étudiées : un environnement répliqué et une décomposition du domaine. Dans ce projet, les expériences ont été réalisées avec un environnement MPI simulé, ce qui limite les gains de performance observés. Globalement, les résultats montrent que l'approche OpenMP constitue le meilleur compromis entre simplicité d'implémentation et amélioration des performances pour cette application.

A. Structure actuelle du projet

```
src/
   ant.{cpp,hpp}
   ant_vectorized.{cpp,hpp}
   ant_openmp.{cpp,hpp}
   fractal_land.{cpp,hpp}
```

```
   renderer.{cpp,hpp}
   window.{cpp,hpp}
   pheronome.hpp
   rand_generator.hpp
   mpi_stub.h

q1_timing/
   ant_simu.cpp
   ant_simu_headless.cpp
   Makefile

q2_vectorization/
   ant_simu_vectorized.cpp
   ant_simu_vectorized_headless.cpp
   Makefile

q3_openmp/
   ant_simu_openmp.cpp
   ant_simu_openmp_headless.cpp
   Makefile

q4_1_replica/
   ant_simu_mpi_replica.cpp
   Makefile

q4_2_domain/
   ant_simu_mpi_domain.cpp
   Makefile
```